FOODIE

Online Food Ordering & Restaurant Management System

Java Servlets & JSP | Apache Tomcat 9 | PostgreSQL (Neon) | JDBC | MVC Architecture

Full Project Documentation & Viva Guide

Complete walkthrough · Architecture · Database · Code · Commands · 60+ Viva Q&A

Table of Contents

1. Project Introduction

Foodie is a full-featured **online food-ordering and restaurant-management web application**. It lets customers browse a food menu, add items to a cart, apply discount coupons, place orders, pay (mock Cash-on-Delivery / Card / UPI), reserve dining tables, raise complaints and chat with an AI support assistant. Delivery riders pick up and deliver orders using a secure 4-digit PIN, and an administrator manages the entire platform (users, menu items, orders, tables, reservations, coupons and complaints).

The application is built as a **classic Java EE web app** using the **Servlet + JSP** model with a hand-written **JDBC data-access layer** talking to a cloud **PostgreSQL** database. It follows the **MVC (Model-View-Controller)** design pattern and the **Front-Controller** pattern (a single servlet routes every request).

Project Name	Foodie
Type	Dynamic Web Application (Java EE / Jakarta Servlet)
Architecture	MVC + Front Controller + DAO pattern
Roles	Customer (USER), Delivery Rider (RIDER), Administrator (ADMIN)
Server	Apache Tomcat 9.0 (Servlet 4.0 API)
Database	PostgreSQL hosted on Neon (serverless cloud DB)
Language	Java 21
Build	Raw `javac` — no Maven/Gradle

2. Features Overview

Actor	Capabilities
Customer (USER)	Register/login, browse & search menu, cart (add/update/remove/clear), apply coupons, checkout with COD/Card/UPI, track orders (Placed → Accepted → Picked-up → Delivered), cancel orders, rate delivered orders (1–5★), reserve tables (with optional dine-in food order), raise complaints, edit profile (name/phone/photo/password/email), AI chat support.

Rider (RIDER)	See accepted orders available for pickup, claim (pick up) an order, deliver by verifying the customer's 4-digit delivery PIN, view own delivery history and payment method.
Admin (ADMIN)	Dashboard with live metrics, manage users (CRUD + role/tenant), manage menu items (CRUD + image upload/URL), accept/reject orders, verify UPI payment screenshots, manage dining tables, approve/reject reservations, create/toggle/delete coupons, resolve complaints, view feedback.

3. Technology Stack

Layer	Technology	Purpose
Presentation (View)	JSP, HTML5, CSS3, Vanilla JavaScript	User interface, rendered server-side
Controller	Java Servlet (`HttpServlet`)	Request routing & business logic
Model	Plain Java Objects (POJOs / JavaBeans)	Data structures
Data Access	JDBC + DAO pattern	SQL persistence
Database	PostgreSQL 16 (Neon serverless)	Persistent storage
Driver	PostgreSQL JDBC `postgresql-42.7.4.jar`	Java↔DB connectivity
Server	Apache Tomcat 9.0	Servlet container
AI Support	NVIDIA NIM API (OpenAI-compatible, LLaMA-3.1-8B)	Chatbot replies
Deployment	Docker + Railway (cloud)	Containerised hosting
File upload	Servlet `@MultipartConfig`	Item & payment images

4. System Architecture (MVC)

Foodie is a textbook **Model-View-Controller** application layered on top of the **Front-Controller** and **DAO** patterns.

MVC Part	In Foodie	Responsibility
Model	`com.foodie.model.*` + `com.foodie.db.*` (DAOs)	Business data & persistence
View	`WEB-INF/views/*.jsp`	Presentation (HTML generation)
Controller	`com.foodie.web.FoodieServlet`	Receives requests, calls model, selects view

> **Why views live inside `WEB-INF/`:** files under `WEB-INF` cannot be requested directly by a browser. This forces every page to go *through the servlet*, guaranteeing that authentication and business logic always run before a JSP renders.

5. Project Folder Structure

```
foodie/
├── WEB-INF/
│   ├── web.xml                ← Deployment descriptor (servlet & filter mappings)
│   ├── classes/               ← Compiled .class files + db.properties
│   │   └── com/foodie/...
│   ├── lib/
│   │   └── postgresql-42.7.4.jar ← JDBC driver
│   ├── src/com/foodie/        ← Java SOURCE code
│   │   ├── web/               → FoodieServlet.java, JspUrlFilter.java
│   │   ├── db/                → *Dao.java, DatabaseManager.java
│   │   ├── model/             → User, Order, Item, Coupon, ... (POJOs)
│   │   └── chat/              → ChatService.java (AI support)
│   └── views/                 ← JSP pages (the View layer)
│       ├── index.jsp, login.jsp, menu.jsp, cart.jsp, checkout.jsp ...
│       ├── admin/             → dashboard, users, items, orders, coupons ...
│       └── rider/             → dashboard.jsp
├── assets/                    ← Static files (served directly)
│   ├── css/style.css
│   ├── js/script.js, theme.js
│   └── images/                → items/, payments/ (uploads land here)
├── Dockerfile                 ← Container build for Railway
├── entrypoint.sh              ← Binds Tomcat to Railway's $PORT
└── railway.json               ← Railway deploy config
```

> **Package layout:** `com.foodie.web` (controllers/filters), `com.foodie.db` (DAOs + DB connection), `com.foodie.model` (data objects), `com.foodie.chat` (AI service).

6. Request Life-Cycle — How It Works

Example: a logged-in customer clicks **"Menu"**.

1. Browser sends `GET /foodie/menu`.
2. **JspUrlFilter** sees a plain page GET with no extension → redirects to `/menu.jsp` (so the address bar looks like a real page).
3. **web.xml** maps both `/menu` and `/menu.jsp` to `FoodieServlet`.
4. `FoodieServlet.route()` strips the `.jsp` suffix → route key becomes `/menu`.
5. `doGet()`'s `switch` matches `case "/menu"`.
6. `ensureLoggedIn()` confirms a valid session (else redirect to `/login`).
7. `showMenu()` calls `itemDao.findAll()` → runs a `SELECT` on PostgreSQL.
8. Results are put on the request: `req.setAttribute("items", ...)`.
9. The servlet *forwards* to `/WEB-INF/views/menu.jsp`.
10. The JSP loops over the items list and generates HTML, which Tomcat returns to the browser.

> **POST-Redirect-GET (PRG):** After every form POST (login, add-to-cart, place-order), the servlet responds with `res.sendRedirect(...)` instead of forwarding. This prevents the "resubmit form?" browser prompt on refresh and keeps URLs clean.

7. The Front Controller — `FoodieServlet`

A **single servlet handles every dynamic route**. This is the Front-Controller pattern. It extends `HttpServlet` and overrides `doGet()` and `doPost()`, each a big `switch` on the route key.

Routing helper (suffix stripping)

```
private static String route(HttpServletRequest req) {
    String path = req.getServletPath();          // e.g. "/menu.jsp"
    if (path != null && path.endsWith(".jsp")) {
        return path.substring(0, path.length() - ".jsp".length());  // "/menu"
    }
    return path;
}
```

doGet skeleton

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) {
    switch (route(req)) {
        case "/login":     ...                      break;
        case "/menu":      if (!ensureLoggedIn(req,res)) return;
                           showMenu(req, res);        return;
        case "/admin/orders":
                           if (!ensureLoggedIn(req,res)) return;
                           if (!authorizeRole(req,res,"ADMIN")) return;
                           showAdminOrders(req, res);  return;
        default:           showHome(req, res, "");
    }
}
```

The `@MultipartConfig` annotation on the class enables file uploads (item images and UPI payment screenshots) with size limits — max file 5 MB, max request 20 MB.

Guard method	What it does
`ensureLoggedIn()`	Redirects to `/login` if no `userId` in session; returns boolean.
`authorizeRole(req,res,"ADMIN")`	Redirects to `/dashboard` unless the session role matches.
`ensureRoleInSession()`	Recovers the role from the DB if a session lost it.
`route()`	Normalises `.jsp`-suffixed paths back to clean route keys.

8. Clean-URL Filter — `JspUrlFilter`

A **Servlet Filter** mapped to `/*` (every request). Its only job is to make page URLs end in `.jsp` for display, while keeping clean routes internally.

- Only rewrites **GET** requests for pages with **no file extension**.
- Leaves static assets (`.css`, `.js`, `.png`), form POSTs, the AJAX `/chat` call, and the site root untouched.
- Sends a redirect: `/menu` → `/menu.jsp`; the servlet then strips it back.

```
public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) {
    HttpServletRequest req = (HttpServletRequest) request;
    if (shouldAppendJsp(req)) {                    // GET + no extension
        String q = req.getQueryString();
        res.sendRedirect(req.getContextPath() + req.getServletPath() + ".jsp"
                         + (q != null ? "?" + q : ""));
        return;
    }
    chain.doFilter(request, response);             // otherwise continue
}
```

9. Database Layer & Connection

Connections come from a single utility class, `DatabaseManager`. It loads the JDBC driver once in a `static` block and reads credentials from environment variables first, then falls back to `WEB-INF/classes/db.properties`.

```
public final class DatabaseManager {
    static {
        Class.forName("org.postgresql.Driver");         // load JDBC driver once
        url      = firstNonBlank(System.getenv("DB_URL"),      props.getProperty("db.url"));
        username = firstNonBlank(System.getenv("DB_USERNAME"), props.getProperty("db.username"));
        password = firstNonBlank(System.getenv("DB_PASSWORD"), props.getProperty("db.password"));
        ready = true;
    }
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(url, username, password);
    }
}
```

> **Self-bootstrapping schema:** every DAO runs its `CREATE TABLE IF NOT EXISTS` (and `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` migrations) inside its constructor. So the very first time the app runs, all tables are created automatically — *no manual SQL migration step*. The default admin (`admin@gmail.com` / `admin123`) is also auto-created.

db.properties (local fallback)

```
db.url=jdbc:postgresql://<host>.neon.tech:5432/neondb?sslmode=require
db.username=neondb_owner
db.password=********
nvidia.api.url=https://integrate.api.nvidia.com/v1/chat/completions
nvidia.api.key=nvapi-********
nvidia.model=meta/llama-3.1-8b-instruct
```

10. Database Schema (All Tables)

Nine tables, all in PostgreSQL. (The users table is deliberately named `resturent` in code.)

Table	Purpose	Key Columns

`resturent`	Users (all roles)	id, name, email(UNIQUE), password, role, tenant_id, phone, photo_url
`items`	Menu catalog	id, name, category, price, discount, image_path
`orders`	Customer orders	id, order_code, user_id, total, status, rider_id, delivery_pin, coupon_code, discount, payment_proof, rating
`order_items`	Line items of an order	id, order_id(FK→orders), item_id, item_name, price, quantity
`restaurant_tables`	Dining tables	id, table_name, shape, capacity, floor, zone, active
`reservations`	Table bookings	id, reservation_code, user_id, table_id, reserve_date, time_in, time_out, party_size, status, order_id
`coupons`	Discount codes	id, code(UNIQUE), type, value, min_order, expiry_date, active
`complaints`	Customer complaints	id, complaint_code, user_id, order_id, message, status, admin_reply
`feedback`	Public footer messages	id, name, email, message, created_at

Example — the `orders` table (DDL)

```
CREATE TABLE IF NOT EXISTS orders (
  id             SERIAL PRIMARY KEY,
  order_code     VARCHAR(40) UNIQUE,
  user_id        INTEGER,
  customer_name VARCHAR(255),
  tenant_id      INTEGER NOT NULL DEFAULT 1,
  address        VARCHAR(512),
  phone          VARCHAR(40),
  payment_method VARCHAR(20),
  total          NUMERIC(10,2) NOT NULL DEFAULT 0,
  status         VARCHAR(20)  NOT NULL DEFAULT 'PENDING',
  rider_id       INTEGER,
  rider_name     VARCHAR(255),
  delivery_pin   VARCHAR(4),
  table_id       INTEGER,
  table_name     VARCHAR(120),
  coupon_code    VARCHAR(40),
  discount       NUMERIC(10,2) NOT NULL DEFAULT 0,
  payment_proof VARCHAR(255),
  rating         INTEGER,
  created_at     TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS order_items (
  id         SERIAL PRIMARY KEY,
  order_id   INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  item_id    INTEGER,
  item_name VARCHAR(255),
  price      NUMERIC(10,2) NOT NULL DEFAULT 0,
  quantity   INTEGER NOT NULL DEFAULT 1
);
```

Entity relationships

```
resturent (user) 1──∞ orders 1──∞ order_items ∞──1 items
resturent (user) 1──∞ reservations ∞──1 restaurant_tables
orders 1──0..1 reservations          (dine-in link via order_id)
resturent (user) 1──∞ complaints ∞──1 orders
coupons  1──∞ orders                 (applied coupon_code)
```

11. DAO Classes Reference

Each DAO wraps all SQL for one table using **JDBC** with **PreparedStatement** (to prevent SQL injection) and **try-with-resources** (auto-close connections).

UserDao — table `resturent`

Method	SQL / Action
`findByEmail(email)`	`SELECT ... WHERE email = ?` — used at login
`emailExists(email)`	`SELECT 1 ... WHERE email = ? LIMIT 1`
`createUserWithRoleTenant(...)`	`INSERT INTO resturent (name,email,password,role,tenant_id)`
`updatePassword(email,new)`	`UPDATE ... SET password = ? WHERE email = ?`
`updateProfile / updateEmail`	profile edits; `updateEmail` throws if email already in use
`findAll / findById / deleteById / updateRoleTenantById`	admin user management

> `ensureDefaultAdminUser()` guarantees `admin@gmail.com` exists with role ADMIN.

OrderDao — tables `orders` + `order_items`

Method	SQL / Action
`createOrder(order, lines)`	**Transactional** — insert order + batch-insert items, commit or rollback
`updateStatus(id, ACCEPTED)`	admin accept: sets status + a unique 4-digit `delivery_pin`, guarded `WHERE status='PENDING'`
`claimOrder(id, riderId, name)`	race-safe: `WHERE rider_id IS NULL AND status='ACCEPTED'` → PICKED_UP
`markDelivered(id, riderId, pin)`	`WHERE ... AND delivery_pin = ?` — PIN check is atomic in SQL
`cancelOrder(id, userId)`	customer cancel, only when PENDING/ACCEPTED
`rateOrder(id, userId, 1..5)`	only when status='DELIVERED'

`findAvailableForRider()`	`WHERE status='ACCEPTED' AND rider_id IS NULL`
`countByStatus(status)`	dashboard metrics

Order status constants: PENDING ACCEPTED PICKED_UP DELIVERED REJECTED CANCELLED

Other DAOs (summary)

DAO	Table	Notable methods / logic
ItemDao	items	findAll, findById, countAll, createItem, updateItem, deleteById
TableDao	restaurant_tables	findAllActive, createTable, updateTable, deleteById
ReservationDao	reservations	`isTableAvailable()` uses interval-overlap check; create, cancel, updateStatus, linkOrder
CouponDao	coupons	`findUsable(code, subtotal)` validates active + not-expired + min-order in one SQL query; handles duplicate code via SQLState 23505
ComplaintDao	complaints	create, findByUserId, resolve (OPEN→RESOLVED with reply)
FeedbackDao	feedback	create, findRecent(n), countAll

Typical DAO method — PreparedStatement + try-with-resources

```
public Item findById(int id) throws SQLException {
    String sql = "SELECT id,name,category,price,discount,image_path FROM items WHERE id = ?";
    try (Connection c = DatabaseManager.getConnection();
         PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setInt(1, id);                       // ? bound safely — no SQL injection
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return new Item(rs.getInt("id"), rs.getString("name"), ...);
            }
        }
    }
    return null;
}
```

12. Model (POJO) Classes

Plain data classes (JavaBeans): private fields + public getters/setters + a no-arg constructor, so JSP Expression Language can read them.

Class	Represents	Interesting behaviour
User	Any account	role, tenantId, phone, photoUrl
Item	Menu dish	price + discount%
Order	An order	list of OrderItem, deliveryPin, rating, paymentProof
OrderItem	One order line	`getLineTotal()` = price × quantity
Coupon	Discount code	`discountFor(subtotal)` : PERCENT or FLAT, capped at subtotal

Reservation	Table booking	status lifecycle, optional orderId
DiningTable	Physical table	shape, capacity, floor, zone, active
Complaint / Feedback	Support messages	status, adminReply

```
// Coupon core logic
public double discountFor(double subtotal) {
    double d = PERCENT.equals(type) ? subtotal * (value / 100.0) : value;
    if (d < 0)        d = 0;
    if (d > subtotal) d = subtotal;   // never discount more than the bill
    return d;
}
```

13. Authentication & Role-Based Access Control

Authentication is **session-based**. On successful login the servlet stores the identity in the `HttpSession`:

```
HttpSession session = req.getSession(true);
session.setAttribute("userId",   user.getId());
session.setAttribute("userName", user.getName());
session.setAttribute("userEmail",user.getEmail());
session.setAttribute("userRole", normalizedRole);   // USER / RIDER / ADMIN
session.setAttribute("tenantId", user.getTenantId());
```

- **ensureLoggedIn** protects customer pages.
- **authorizeRole(...,"ADMIN"/"RIDER")** protects admin/rider pages (RBAC).
- **Logout** calls `session.invalidate()`.
- After login, the user resumes the page they were bounced from (`afterLogin`), else the dashboard.
- Customers land on `/menu`, admins on `/admin/dashboard`, riders on `/rider/dashboard`.

> **Honest limitation (good to mention in viva):** passwords are stored as *plain text* in this student project. In production you would hash them with **BCrypt**/PBKDF2 and add a salt. This is the single biggest security improvement to name if asked.

14. Functional Modules (Deep Dive)

14.1 Cart (session-based)

The cart is a `Map<Integer,Integer>` (itemId → quantity) kept in the session — no DB writes until checkout. Actions: `add` (merge quantities), `update`, `remove`, `clear`.

```
cart.merge(itemId, qty, Integer::sum);   // add: increment existing quantity
```

14.2 Checkout & Coupons

- Prices are **re-fetched from the DB** at checkout — client-side prices are never trusted.

- Applied coupons are **re-validated server-side** against the real subtotal before the order is saved.
- Payment methods: **COD**, **Card**, **UPI** (UPI requires a screenshot upload).
- On success the cart is cleared and the user is redirected to `/orders?placed=ORD...` with the receipt auto-opened.

14.3 Order Lifecycle

```
Customer places  →  PENDING
Admin accepts    →  ACCEPTED   (a unique 4-digit delivery PIN is generated)
Rider picks up   →  PICKED_UP  (claimOrder: race-safe, one rider only)
Rider delivers   →  DELIVERED  (must enter the customer's correct PIN)
Admin rejects    →  REJECTED
Customer cancels →  CANCELLED  (only while PENDING or ACCEPTED)
```

> **Delivery PIN security:** the PIN is verified inside the SQL `WHERE` clause (`markDelivered`), so a wrong PIN simply updates 0 rows — no way to bypass it from Java.

14.4 Table Reservation

Customer picks a date + time window and a table. Availability uses an **interval-overlap test** (`existing.time_in < requested.time_out AND existing.time_out > requested.time_in`). A booking can optionally continue into a **dine-in food order** (the table stands in for a delivery address).

14.5 Admin Dashboard

Aggregates live metrics: total items, total orders, counts per status, feedback count, open complaints and active coupons — all via `COUNT(*)` queries.

15. AI Chat Support Module

`ChatService` is a server-side client for **NVIDIA's OpenAI-compatible chat API** (model LLaMA-3.1-8B). The browser only ever talks to our own `POST /chat` endpoint, so the API key **never reaches the client**.

- Uses Java 11's built-in `java.net.http.HttpClient` — no external HTTP library.
- A fixed **system prompt** constrains the bot to Foodie-related help.
- Login is required; input is capped at 1000 chars.
- JSON is built and parsed by hand (no JSON library on the classpath): `jsonEscape()` and `extractContent()`.
- Never throws — returns a friendly fallback string on any failure.

```
String reply = ChatService.getInstance().reply(message);
res.getWriter().write("{\"reply\":\"" + ChatService.jsonEscape(reply) + "\"}");
```

16. Security Aspects

Concern	Mitigation in Foodie

SQL Injection	**PreparedStatement** with bound `?` parameters everywhere
XSS (cross-site scripting)	`escape()` HTML-encodes user text (&, <, >, ", ')
Direct JSP access	JSPs live under `WEB-INF/` → unreachable without the servlet
Broken access control	`authorizeRole()` guards every admin/rider route (RBAC)
Price / discount tampering	Prices & coupon discounts recomputed server-side from the DB
Race conditions	Status transitions guarded in SQL `WHERE` (claim, deliver, cancel)
Delivery fraud	4-digit PIN checked atomically in SQL
Secret leakage	DB & API credentials read from env vars / server-side properties, never sent to browser
Upload abuse	`@MultipartConfig` size limits (5 MB file / 20 MB request)

> **Known weaknesses to acknowledge:** plain-text passwords, secrets committed in `db.properties`, no CSRF tokens, session-fixation not hardened. All are standard "future work" talking points.

17. Important Syntax Explained

Servlet declaration

```
public class FoodieServlet extends HttpServlet {
    protected void doGet (HttpServletRequest req, HttpServletResponse res) { ... }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) { ... }
}
```

Forward vs Redirect

```
// FORWARD — server-side, same request, URL unchanged, can pass attributes
req.getRequestDispatcher("/WEB-INF/views/menu.jsp").forward(req, res);

// REDIRECT — tells browser to make a NEW request, URL changes (used after POST)
res.sendRedirect(req.getContextPath() + "/orders");
```

Session handling

```
HttpSession s = req.getSession(true);   // create if none
s.setAttribute("userId", 42);
Object id = s.getAttribute("userId");
s.invalidate();                         // logout
```

JDBC pattern (the “big 5”)

```
Connection con  = DriverManager.getConnection(url, user, pass);
PreparedStatement ps = con.prepareStatement("SELECT * FROM items WHERE id = ?");
ps.setInt(1, id);
ResultSet rs = ps.executeQuery();       // executeUpdate() for INSERT/UPDATE/DELETE
while (rs.next()) { rs.getString("name"); }
```

JSP + Expression Language + scriptlet

```
<%@ page import="java.util.List, com.foodie.model.Item" %>
<% List<Item> items = (List<Item>) request.getAttribute("items"); %>
<% for (Item it : items) { %>
    <h3><%= it.getName() %></h3>            <%-- <%= %> prints a value --%>
<% } %>
Context path via EL: ${pageContext.request.contextPath}
```

web.xml mapping

```
<servlet>
  <servlet-name>FoodieServlet</servlet-name>
  <servlet-class>com.foodie.web.FoodieServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>FoodieServlet</servlet-name>
  <url-pattern>/menu</url-pattern>
  <url-pattern>/menu.jsp</url-pattern>
</servlet-mapping>
```

18. Build, Run & Deploy Commands

Compile (no Maven/Gradle — raw javac, run from `WEB-INF/src`)

```
javac -encoding UTF-8 \
  -cp "../lib/postgresql-42.7.4.jar;<tomcat>/lib/servlet-api.jar" \
  -d ../classes \
  com/foodie/model/*.java com/foodie/db/*.java \
  com/foodie/chat/*.java  com/foodie/web/*.java
```

Classes are written to `WEB-INF/classes`; the servlet-api.jar is Tomcat’s own, the Postgres driver is in `WEB-INF/lib`. All four packages must be compiled together.

Start Tomcat (Windows, JDK 21)

```
cd <tomcat>/bin
set "JAVA_HOME=C:\Program Files\Java\jdk-21.0.10"
startup.bat        # (shutdown.bat to stop)
```

Hot-reload after recompiling

```
# Tomcat reloads the context when web.xml changes:
touch WEB-INF/web.xml     # recompile FIRST, then touch (≈10s reload cycle)
```

Run the app

```
http://localhost:8080/foodie/        # welcome page (home)
Login: admin@gmail.com / admin123    # default admin
```

Docker / Railway deploy

```
FROM tomcat:9.0-jre21-temurin        # Dockerfile
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY . /usr/local/tomcat/webapps/ROOT/     # serve at root context "/"
# entrypoint.sh rewrites Tomcat's port to Railway's $PORT, then runs catalina.sh
```

Handy Git commands

```
git add .
git commit -m "Add project documentation"
git push origin main
```

19. Viva Questions & Answers

A. Basics & Architecture

Q1. What is Foodie?

> A Java Servlet/JSP web application for online food ordering and restaurant management, supporting three roles — customer, delivery rider and admin — backed by a PostgreSQL database.

Q2. Which design pattern does the project follow?

> Primarily **MVC**. It also uses the **Front-Controller** pattern (one servlet routes all requests), the **DAO** pattern (one class per table for persistence), and the **Singleton** pattern (ChatService, DatabaseManager).

Q3. What is MVC and how is it applied here?

> Model = POJOs + DAOs (`com.foodie.model` / `db`); View = JSP pages in `WEB-INF/views` ; Controller = `FoodieServlet` . The controller receives the request, uses the model to fetch/save data, then forwards to a view that renders HTML.

Q4. What is a Front Controller? Why use one?

> A single entry-point servlet that handles every request and dispatches to the right handler. Benefits: centralised authentication, consistent routing, and no need to write a separate servlet per page.

Q5. What is a Servlet?

> A Java class running inside a web container (Tomcat) that handles HTTP requests and generates responses. It extends `HttpServlet` and overrides `doGet()` / `doPost()`.

Q6. What is the Servlet life-cycle?

> Three methods managed by the container: `init()` (once, on load) → `service()` (per request, dispatches to doGet/doPost) → `destroy()` (once, on unload). The container creates a single servlet instance and serves concurrent requests with threads.

Q7. What is JSP and how does it relate to servlets?

> JSP (JavaServer Pages) is an HTML template with embedded Java. At runtime the container **compiles each JSP into a servlet**. We use JSP purely for the view layer.

Q8. Difference between forward and redirect?

> **Forward** is server-side within the same request (URL stays the same, attributes are preserved) — used to render a JSP. **Redirect** asks the browser to issue a brand-new request (URL changes) — used after POST to implement Post-Redirect-Get.

Q9. What is the POST-Redirect-GET pattern and why use it?

> After processing a POST, respond with a redirect to a GET page. This prevents duplicate form submissions when the user refreshes and keeps the browser URL clean. Foodie uses it after login, add-to-cart, checkout, etc.

Q10. What is `web.xml`?

> The deployment descriptor. It declares the servlet, the filter and all URL mappings, plus the welcome file. Foodie maps each route both cleanly (`/menu`) and with a `.jsp` suffix.

B. Database & JDBC

Q11. What is JDBC?

> Java Database Connectivity — the standard API for connecting Java to relational databases and running SQL. Core objects: `Connection`, `Statement` / `PreparedStatement`, `ResultSet`.

Q12. Why PreparedStatement instead of Statement?

> It pre-compiles the SQL and binds parameters with `?`, which (a) prevents **SQL injection** and (b) is faster for repeated queries. Foodie uses it for every query.

Q13. What is the DAO pattern?

> Data Access Object — a class that encapsulates all database operations for one entity, so business logic never writes raw SQL. Foodie has UserDao, OrderDao, ItemDao, etc.

Q14. How are database tables created?

> Each DAO runs `CREATE TABLE IF NOT EXISTS` in its constructor (self-bootstrapping). New columns are added with `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`. No manual migration is needed.

Q15. Which database is used and why?

> PostgreSQL, hosted on Neon (serverless cloud). It's free-tier friendly, supports SSL and works from anywhere, which suits cloud deployment on Railway.

Q16. How is the DB connection configured?

> `DatabaseManager` loads the driver once in a static block and reads the URL/username/password from environment variables first, then `db.properties`. `getConnection()` returns a fresh `Connection`.

Q17. What is try-with-resources and why is it used?

> A Java construct that auto-closes resources (Connection, PreparedStatement, ResultSet) at the end of the block, preventing connection leaks. Every DAO method uses it.

Q18. How does the order creation stay consistent?

> `createOrder()` runs a **transaction**: it turns off auto-commit, inserts the order and all line items, then `commit()`s — or `rollback()`s on any error, so you never get an order with missing items.

Q19. What is `SERIAL PRIMARY KEY`?

> A PostgreSQL auto-incrementing integer primary key — the DB assigns a unique id automatically on insert.

Q20. How do you get the generated id back after an insert?

> Prepare the statement with `RETURN_GENERATED_KEYS` and read it from `getGeneratedKeys()`. Foodie does this for orders, reservations and complaints.

C. Security

Q21. How is SQL injection prevented?

> All SQL uses PreparedStatement with bound parameters, so user input is treated as data, never as executable SQL.

Q22. How is XSS handled?

> User-supplied text is HTML-escaped by the `escape()` helper before being written into pages (converting `< > & " '`).

Q23. Why are JSPs under WEB-INF?

> Files under `WEB-INF` can't be opened directly by a browser. This forces every page through the servlet, so authentication and authorization always run first.

Q24. How is role-based access enforced?

> The role is stored in the session at login. Admin/rider routes call `authorizeRole()`, which redirects away anyone whose session role doesn't match.

Q25. Biggest security weakness and how to fix it?

> Passwords are stored in plain text. Fix by hashing with **BCrypt** (salted, slow hash) and comparing hashes at login. Also move secrets out of `db.properties` into environment variables and add CSRF tokens.

Q26. How is the delivery PIN kept secure?

> It's generated uniquely when the admin accepts an order and verified inside the SQL WHERE clause on delivery — a wrong PIN updates zero rows, so it can't be bypassed in Java.

Q27. How do you stop price/coupon tampering?

> The server ignores any price sent by the browser. At checkout it re-reads item prices from the DB and re-validates the coupon against the true subtotal before saving the order.

D. Features & Logic

Q28. How does the cart work?

> It's a `Map<itemId,quantity>` stored in the HttpSession. Nothing is written to the DB until checkout, so it's fast and per-user.

Q29. Walk through the order lifecycle.

> PENDING (placed) → ACCEPTED (admin, PIN generated) → PICKED_UP (rider claims) → DELIVERED (rider enters PIN). Alternatives: REJECTED (admin) or CANCELLED (customer, only while pending/accepted).

Q30. How is a rider stopped from claiming the same order twice?

> `claimOrder()` updates `WHERE rider_id IS NULL AND status='ACCEPTED'`. Only the first rider's update succeeds; a second attempt affects zero rows.

Q31. How is table double-booking prevented?

> Before booking, `isTableAvailable()` runs an interval-overlap query for the same table and date against PENDING/ACCEPTED reservations. If any overlap exists, the booking is refused.

Q32. How do coupons work?

> Admins create PERCENT or FLAT coupons with an optional min-order and expiry. `findUsable()` validates active + not-expired + min-order in one SQL query; `discountFor()` computes the amount, capped at the subtotal.

Q33. What payment methods exist?

> Cash on Delivery, Card, and UPI (mock). UPI additionally requires uploading a payment screenshot, which the admin later verifies.

Q34. How does file upload work?

> The servlet is annotated `@MultipartConfig`. Uploaded images are saved under `assets/images/items` or `/payments` with a unique timestamped filename; the DB stores the relative path.

Q35. What is dine-in ordering?

> A reservation can flow straight into a food order; the table name replaces the delivery address, and the finished order is linked back to the reservation via `linkOrder()`.

Q36. How does the AI chat work without leaking the API key?

> The browser calls our own `/chat` endpoint; the server (`ChatService`) forwards the message to NVIDIA's API with the secret key held server-side and returns only the reply text as JSON.

Q37. What is a Filter and what does JspUrlFilter do?

> A Filter intercepts requests before they reach a servlet. `JspUrlFilter` redirects extension-less page GETs to their `.jsp` form so the address bar shows `.jsp`, without changing any internal routing.

Q38. How are dashboard metrics computed?

> With `COUNT(*)` queries per status/entity (orders by status, item count, feedback count, open complaints, active coupons), gathered in `showAdminDashboard()`.

E. Java & Tooling

Q39. What is a POJO / JavaBean?

> A Plain Old Java Object with private fields, public getters/setters and a no-arg constructor. Foodie's model classes are JavaBeans so JSP EL can read their properties.

Q40. What is the Singleton pattern? Where is it used?

> A class with a single shared instance. `ChatService` exposes `getInstance()`; `DatabaseManager` is effectively a static singleton (private constructor, static methods).

Q41. How is the project built without Maven?

> Directly with `javac`, pointing the classpath at Tomcat's `servlet-api.jar` and the Postgres driver, compiling all four packages into `WEB-INF/classes`.

Q42. How do you deploy a new build to a running Tomcat?

> Recompile the classes, then `touch WEB-INF/web.xml`; Tomcat detects the descriptor change and reloads the web-app context automatically.

Q43. What is a WAR / context path?

> A WAR is a packaged web-app. The context path is the URL prefix for the app — here `/foodie` locally, or root `/` in the Docker/Railway deployment.

Q44. What does `request.setAttribute` vs a session attribute mean?

> A request attribute lives for one request (used to pass data to a JSP during forward); a session attribute persists across requests for one user (used for login identity and the cart).

Q45. What is Expression Language (EL) in JSP?

> A concise syntax like `${pageContext.request.contextPath}` to access objects/properties without Java scriptlets.

Q46. How does the app run in the cloud?

> A Dockerfile builds a Tomcat-9-on-JRE-21 image, copies the app to the root context, and `entrypoint.sh` binds Tomcat to Railway's `$PORT`. `railway.json` configures the build and restart policy.

Q47. What are multi-tenancy hints in the schema (`tenant_id`)?

> A `tenant_id` column exists on users and orders to support multiple restaurants/branches in future; currently it defaults to 1.

Q48. How would you scale or improve this project?

> Add a connection pool (HikariCP), hash passwords, add CSRF protection, move secrets to env vars, introduce a service layer, add REST/JSON APIs, unit tests, and real payment-gateway integration.

Q49. Why store the delivery address as the table name for dine-in?

> Because a dine-in order has no delivery address; using "Dine-in — Table X" keeps a single, uniform order schema for both delivery and dine-in.

Q50. What happens on logout?

> `session.invalidate()` destroys the session (clearing identity and cart), then the user is redirected to the home page.

F. Rapid-fire one-liners

Question	Answer
Server used?	Apache Tomcat 9
Servlet API version?	4.0 (Java EE / `javax.servlet`)
Java version?	21
Database?	PostgreSQL (Neon cloud)
JDBC driver?	postgresql-42.7.4.jar

Default admin?	admin@gmail.com / admin123
Roles?	USER, RIDER, ADMIN
Cart storage?	HttpSession (Map itemId→qty)
Password storage?	Plain text (improvement: BCrypt)
Coupon types?	PERCENT, FLAT
Order statuses?	PENDING, ACCEPTED, PICKED_UP, DELIVERED, REJECTED, CANCELLED
Users table name?	`resturent` (misspelt on purpose)
AI model?	NVIDIA LLaMA-3.1-8B-instruct
Deployment?	Docker image on Railway
Build tool?	None — raw `javac`

20. Glossary of Key Terms

Term	Meaning
Servlet	Java class handling HTTP requests inside a web container
JSP	HTML template with embedded Java, compiled to a servlet
MVC	Model-View-Controller separation of concerns
Front Controller	Single servlet routing all requests
DAO	Data Access Object — class encapsulating a table's SQL
JDBC	Java Database Connectivity API
PreparedStatement	Pre-compiled parameterised SQL (prevents injection)
HttpSession	Per-user server-side state across requests
Filter	Interceptor that runs before servlets
Forward	Server-side dispatch within one request
Redirect	Browser makes a new request to a new URL
PRG	Post-Redirect-Get anti-duplicate-submit pattern
RBAC	Role-Based Access Control
POJO / JavaBean	Plain data object with getters/setters
Context path	URL prefix of the deployed web-app
Deployment descriptor	`web.xml` configuration file